códigofacilito

Ramsés Camas · Bootcamp AI Engineer

Testing & Reliability para AI Systems

Clase 14 · AI Engineer Bootcamp

códigofacilito

Agenda

Clase 14

/01 Fallos silenciosos

Por qué los AI systems se rompen diferente

/02 Datasets

Curados + sintéticos para cobertura real

/03 5 niveles de testing

Contratos, goldens, replay, regresión, evals

/04 CI local

Runner + Makefile, sin servicios externos

El problema fundamental 🎯

Un sistema tradicional es determinista: misma entrada, misma salida. Un AI system es estocástico: misma entrada, salida diferente cada vez. No puedes hacer assertEqual(output, expected) directamente. Necesitas nuevas estrategias de testing que acepten variabilidad controlada.

códigofacilito

Modos de fallo silenciosos

El LLM cambia de versión y las respuestas mutan sin aviso. Un prompt que funcionaba deja de funcionar tras un refactor. El retriever devuelve chunks irrelevantes por un cambio en el índice. El agente entra en loop infinito porque una herramienta devolvió un error inesperado.

códigofacilito

Taxonomía de pruebas

Nivel 1 — Contratos: ¿el output tiene la forma correcta? Schema, tipos, campos requeridos.
Nivel 2 — Golden tests: ¿es semánticamente equivalente a una referencia conocida?
Nivel 3 — Replay: ¿el sistema reproduce comportamientos pasados exitosos?

- Nivel 4 — Regresión: ¿un cambio degradó la calidad? Antes vs. después.
- Nivel 5 — Evaluación offline: ¿cumple métricas sobre un dataset completo?
- Cada nivel atrapa un tipo de fallo distinto. Los cinco juntos dan cobertura real.

códigofacilito

Nivel 1: Pruebas de contratos 🎯

Un contrato define la estructura esperada del output del LLM. No evalúa si la respuesta es buena, solo si es parseable y completa. Es la primera línea de defensa: si el contrato falla, nada downstream funciona. Usamos los modelos Pydantic que ya definimos en la Clase 4 (contracts/).

```python
// Ejemplo de modelo de contrato


# contracts/schemas.py

from pydantic import BaseModel, Field

from typing import List


class DocAnswer(BaseModel):

    answer: str = Field(min_length=10)

    sources: List[str] = Field(min_length=1)

    confidence: float = Field(ge=0.0, le=1.0)
```

```python
// Test de contrato con pytest
# tests/test_contracts.py
import pytest
from contracts.schemas import DocAnswer
def test_valid_doc_answer():
    data = {"answer": "El proceso de RAG...",
            "sources": ["doc_001.pdf"],
            "confidence": 0.85}
    result = DocAnswer(**data)
    assert result.confidence == 0.85
def test_missing_sources_fails():
    with pytest.raises(Exception):
        DocAnswer(answer="Algo valido.", sources=[], confidence=0.5)
```

```python
// Contratos contra el LLM real
# tests/test_llm_contracts.py
import json
from core.llm_client import call_llm
from contracts.schemas import DocAnswer


def test_llm_returns_valid_contract():
    raw = call_llm(
        prompt="Responde en JSON: {answer, sources, confidence}",
        query="Que es RAG?")
    parsed = json.loads(raw)
    result = DocAnswer(**parsed)
    assert result.answer
```

Nivel 2: Golden tests 🎯

Un golden test compara la salida actual contra una referencia aprobada manualmente. No exige igualdad exacta (el LLM no es determinista). Usa métricas de similitud: cosine similarity de embeddings, ROUGE, o keywords obligatorias. Flujo: generar, aprobar manualmente, guardar como golden, comparar en CI.

```
// Estructura del golden file
// evals/golden/rag_questions.json
[
  {
    "id": "q001",
    "query": "Cual es la politica de vacaciones?",
    "context_doc": "politica_rrhh.pdf",
    "golden_answer": "La empresa ofrece 15 dias habiles.",
    "required_keywords": ["15 dias", "vacaciones"],
    "min_similarity": 0.75
  }
]
```

```python
# evals/golden_test.py
from sentence_transformers import SentenceTransformer
from sklearn.metrics.pairwise import cosine_similarity
model = SentenceTransformer("all-MiniLM-L6-v2")
def check_golden(actual, golden, keywords, min_sim):
    emb = model.encode([actual, golden])
    sim = cosine_similarity([emb[0]], [emb[1]])[0][0]
    kw_hits = [k for k in keywords if k.lower() in actual.lower()]
    return {"similarity": round(sim, 3),
            "sim_pass": sim >= min_sim,
            "kw_pass": len(kw_hits) == len(keywords)}
```

```
// Golden tests con pytest
# tests/test_goldens.py
import json, pytest
from evals.golden_test import check_golden
from core.llm_client import call_llm
with open("evals/golden/rag_questions.json") as f:
    GOLDENS = json.load(f)
@pytest.mark.parametrize("case", GOLDENS,
    ids=[c["id"] for c in GOLDENS])
def test_golden(case):
    actual = call_llm(prompt=case["query"])
    result = check_golden(actual, case["golden_answer"],
        case["required_keywords"], case["min_similarity"])
    assert result["sim_pass"]
    assert result["kw_pass"]
```

códigofacilito

Cuándo actualizar los goldens

Actualiza cuando cambias el prompt intencionalmente, migras de modelo, o rediseñas el pipeline. No actualices cuando el test falla por una regresión inesperada: investiga primero.

- **Workflow: (1) Correr tests. (2) Si falla, revisar si el cambio fue intencional. (3) Si sí, regenerar y aprobar golden. (4) Commit del nuevo golden con mensaje explicativo. Trátalo como snapshot test: el golden es la verdad aprobada por un humano.**

códigofacilito

Nivel 3: Replay de trazas 🎯

Una traza es el registro completo de una ejecución exitosa: inputs, outputs intermedios, decisiones del agente, herramientas llamadas, respuestas del LLM. Replay es volver a ejecutar el sistema con los mismos inputs y verificar que el comportamiento es equivalente. Útil para detectar regresiones tras cambios en prompts, herramientas o versiones de modelo.

```python
// Capturar una traza
# evals/trace_capture.py
import json, datetime
def capture_trace(trace_id, steps):
    trace = {
        "id": trace_id,
        "timestamp": datetime.datetime.now().isoformat(),
        "steps": steps
    }
    path = f"evals/traces/{trace_id}.json"
    with open(path, "w") as f:
        json.dump(trace, f, indent=2, ensure_ascii=False)
    return path
```

```
// Ejecutar el replay

# evals/trace_replay.py

import json

from evals.golden_test import check_golden

def replay_trace(trace_path, system_fn):

    with open(trace_path) as f:

        trace = json.load(f)

    original_steps = trace["steps"]

    query = original_steps[0]["input"]

    new_output = system_fn(query)

    original_final = original_steps[-1]["output"]

    result = check_golden(new_output, original_final,

                          keywords=[], min_sim=0.70)

    return {"trace_id": trace["id"],"pass": result["sim_pass"]}
```

códigofacilito

Cuándo usar replay de trazas

- **Regla práctica: mantén 20 a 50 trazas representativas curadas. No necesitas miles. Menos es más cuando cada traza aporta cobertura única. La calidad de las trazas pesa más que la cantidad bruta.**

Antes de desplegar, corres replay de las 20 trazas más críticas. Tras cambio de modelo, verificas que el comportamiento no diverge. Tras refactor de agente, confirmas que la lógica de herramientas sigue el mismo patrón.

Nivel 4: Regresión

🎯

Un cambio en el system prompt, en la descripción de una herramienta o en el formato de un template puede romper todo silenciosamente. Test de regresión es antes vs después: corres el mismo dataset con el prompt anterior y el nuevo, y comparas métricas agregadas. Si el delta es negativo, el cambio degradó calidad.

```python
// Implementación de regresión
# evals/regression.py
from evals.golden_test import check_golden
def run_regression(dataset, fn_old, fn_new):
    results = []
    for case in dataset:
        old_out = fn_old(case["query"])
        new_out = fn_new(case["query"])
        old_s = check_golden(old_out, case["golden_answer"], [], 0.0)
        new_s = check_golden(new_out, case["golden_answer"], [], 0.0)
        results.append({"id": case["id"],
            "delta": new_s["similarity"] - old_s["similarity"]})
    return results
```

Criterios de aceptación

Regla individual: Ningún caso baja más de 0.15 en similitud. Regla agregada: El promedio no baja más de 0.05. Regla de keywords: El porcentaje de keywords encontradas no disminuye.

Si cualquier regla se viola, el cambio no pasa. Investiga antes de mergear. La regresión silenciosa es el peor enemigo: se acumula a lo largo de semanas hasta que el sistema degrada sin una causa identificable.

códigofacilito

Datasets de evaluación 🎯

Sin dataset, tus tests son anecdóticos: probé con 3 preguntas y funcionó. Un dataset curado te da cobertura medible y repetible. Tipos: curado manualmente (alta calidad, costoso) y sintético (generado por LLM, alto volumen, menor garantía). Ideal: combinar ambos. 30 a 50 casos curados más 200 sintéticos.

códigofacilito

Datasets de evaluación

Sin dataset, tus tests son anecdóticos: probé con 3 preguntas y funcionó. Un dataset curado te da cobertura medible y repetible. Tipos: curado manualmente (alta calidad, costoso) y sintético (generado por LLM, alto volumen, menor garantía).

- **Ideal: combinar ambos. 30 a 50 casos curados más 200 sintéticos. Los curados marcan la línea base de calidad no negociable. Los sintéticos dan amplitud para detectar fallos borde que no habías imaginado.**

```
// Estructura de dataset curado
# evals/datasets.py
CURATED_DATASET = [
  {
    "id": "c001",
    "query": "Cuantos dias de vacaciones tengo?",
    "category": "factual",
    "difficulty": "easy",
    "expected_source": "politica_rrhh.pdf",
    "golden_answer": "15 dias habiles al ano.",
    "required_keywords": ["15", "dias", "habiles"],
    "min_similarity": 0.75
  },
]
```

```
// Generación sintética con LLM

# evals/synthetic_gen.py

from core.llm_client import call_llm

import json

def generate_synthetic(doc_chunk, n=5):

    prompt = f"""Dado este fragmento, genera {n} preguntas

    que un usuario haria, con su respuesta esperada.

    Responde SOLO en JSON:

    [{{"query": "...", "answer": "..."}}]


    Documento:

    {doc_chunk}"""

    raw = call_llm(prompt=prompt)

    return json.loads(raw)
```

códigofacilito

Validación de sintéticos

Nunca uses sintéticos sin revisión humana. Workflow: generar 200, revisar 50 manualmente. Si más del 80% son correctos, el lote es usable.

- **Marcar cada caso como verified: true/false. Los verificados entran al dataset de evaluación oficial. Los no verificados sirven para smoke tests y cobertura exploratoria rápida sin compromiso de calidad.**

```python
// El runner de evaluación

# evals/runner.py

import json

from evals.golden_test import check_golden

from evals.datasets import CURATED_DATASET
```

```python
def run_eval(system_fn, dataset=None):

    dataset = dataset or CURATED_DATASET

    results = []

    for case in dataset:

        actual = system_fn(case["query"])

        score = check_golden(actual, case["golden_answer"],

            case.get("required_keywords", []),

            case.get("min_similarity", 0.70))

        results.append({"id": case["id"], **score})

    passing = sum(1 for r in results if r["sim_pass"])

    print(f"Pass rate: {passing}/{len(results)}")

    return results
```

```
// CI mínima local con Makefile

# Makefile

.PHONY: test test-contracts test-goldens eval

test-contracts:

      python -m pytest tests/test_contracts.py -v

test-goldens:

      python -m pytest tests/test_goldens.py -v

test:

      python -m pytest tests/ -v

eval:

      python -c "from evals.runner import run_eval; \

                 from my_system import answer; run_eval(answer)"

pre-deploy: test eval

      @echo "Todos los tests pasaron."
```

códigofacilito

Tu arsenal de testing

Contratos → ¿la forma es correcta? Pydantic + pytest. Golden tests → ¿la sustancia es correcta? Embeddings + keywords. Replay de trazas → ¿el comportamiento se mantiene? Re-ejecución + comparación.

- **Regresión → ¿el cambio mejoró o empeoró? Antes vs. después. Datasets → ¿tengo cobertura real? Curados + sintéticos. Runner + Makefile → ¿puedo correr todo con un comando? make pre-deploy.**

códigofacilito

*Solo da doble click para cambiar la imagen

Artefactos de hoy y próxima clase 🎯

Próxima clase (15): Evals para RAG y agentes con DSPy como toolbox de optimización. Usaremos RAGAS, faithfulness, precision, recall y compilaremos componentes con DSPy para mejorar automáticamente prompts y retrieval.